

Relatório de Aplicação dos Princípios SOLID no Código do Projeto “ObservAção”

Análise das melhorias estruturais e de manutenção a partir da aplicação dos princípios SOLID

Introdução

Os princípios SOLID são um conjunto de boas práticas de design de software que têm como objetivo tornar o código mais organizado, modular e de fácil manutenção. Esses princípios orientam a forma como classes e métodos devem ser estruturados, promovendo baixo acoplamento, alta coesão e maior flexibilidade para evolução do sistema.

O projeto ObservAção consiste em um sistema de gestão de solicitações cidadãos, desenvolvido para facilitar a comunicação entre a população e o poder público. A aplicação permite que cidadãos registrem demandas relacionadas a serviços públicos, como problemas urbanos e necessidades da comunidade, possibilitando o acompanhamento dessas solicitações por meio de um protocolo e a atualização de seu status ao longo do atendimento. Além do cidadão, o sistema também envolve o servidor público, responsável por visualizar as solicitações registradas, organizar a fila de atendimento e atualizar o status das demandas, permitindo que cada solicitação avance até sua resolução.

Aplicação dos Princípios SOLID

1. Single Responsibility Principle

O princípio da responsabilidade única foi aplicado no projeto ao dividir o sistema em classes com papéis bem definidos, evitando que uma mesma classe assumisse múltiplas funções ao mesmo tempo. Essa separação pode ser observada principalmente na distinção entre classes que representam dados, classes que executam regras de negócio e classes responsáveis pelo fluxo de interação com o usuário.

A classe *Solicitacao* tem como única responsabilidade representar uma solicitação e controlar seu ciclo de vida dentro do sistema. Ela concentra atributos como protocolo, descrição, localização, prioridade, status atual e histórico, sendo responsável por manter a consistência desses dados ao longo do tempo. Isso fica evidente no método:

```

public void atualizarStatus(StatusSolicitacao novoStatus, String
comentario, Usuario usuario){
    HistoricoStatus h = new HistoricoStatus(novoStatus, comentario,
usuario);
    adicionarHistorico(h);
    statusAtual = novoStatus;
}

```

Nesse método, a classe gerencia exclusivamente a mudança de estado da solicitação, garantindo que toda atualização de status gere um novo registro no histórico. Ou seja, ela não apenas armazena dados, mas também controla a regra interna de evolução da solicitação. Ao mesmo tempo, ela não se preocupa com validação de entrada, persistência ou interação com o usuário, mantendo sua responsabilidade isolada.

A classe *HistoricoStatus* complementa essa estrutura ao assumir exclusivamente a responsabilidade de representar cada alteração de status. Ela armazena o status, o comentário, a data da mudança e o responsável. Sua existência evita que a lógica de histórico fique dentro da *Solicitacao*, o que tornaria essa classe mais complexa e com múltiplas responsabilidades. Dessa forma, qualquer alteração na forma como o histórico é registrado impacta apenas essa classe.

A classe *SolicitacoesService* é responsável pelas regras de negócio do sistema. Ela atua como intermediária entre a interface (Menu) e as entidades do domínio. No método *criarSolicitacao*, por exemplo, são realizadas todas as validações necessárias antes da criação do objeto:

```

public Solicitacao criarSolicitacao(Usuario usuario, Categoria
categoria, String descricao, String localizacao, Prioridade
prioridade)

```

Nesse ponto, a responsabilidade do *service* é garantir que os dados estejam corretos antes de delegar a criação da solicitação. Isso evita que a classe *Solicitacao* precise lidar com validações externas, o que quebraria o *Single Responsibility Principle*. Além disso, métodos como *buscarPorProtocolo* e *atualizarStatus* centralizam operações do sistema, evitando que essas regras fiquem espalhadas em diferentes partes do código.

A classe *FilaAtendimento* possui uma responsabilidade bem específica: gerenciar a coleção de solicitações. Ela é responsável por adicionar novas solicitações e fornecer diferentes formas de organização dessas informações. Isso pode ser observado nos métodos:

```

public void adicionarSolicitacao(Solicitacao solicitacao)
public List<Solicitacao> listarPorPrioridade()
public List<Solicitacao> listarPorCategoria()
public List<Solicitacao> listarPorStatus()

```

Cada um desses métodos atua apenas sobre a lista de solicitações, sem interferir na lógica de criação, validação ou atualização. Isso garante que mudanças na forma de organização ou ordenação impactem apenas essa classe.

Já a classe *Menu* concentra a responsabilidade de interação com o usuário. Ela gerencia entrada de dados, navegação entre opções e exibição de informações. Métodos como *iniciar*, *menuCidadao* e *menuServidorPublico* controlam o fluxo do sistema, enquanto métodos como *imprimirLista* e *imprimirTextoFormatado* cuidam da apresentação dos dados.

A separação de responsabilidades adotada no restante do sistema melhora diretamente a manutenção, pois cada parte pode evoluir de forma independente. Alterações em regras de negócio ficam concentradas no *SolicitacoesService*, mudanças na estrutura da solicitação afetam apenas a classe *Solicitacao*, e ajustes na organização dos dados impactam somente a *FilaAtendimento*. Isso reduz o risco de efeitos colaterais e torna o código mais fácil de entender, modificar e expandir.

2. Open/Closed Principle

O princípio do aberto/fechado define que o sistema deve estar aberto para extensão, mas fechado para modificação, ou seja, deve ser possível adicionar novos comportamentos sem precisar alterar código já existente.

No projeto, esse princípio aparece principalmente no uso de enums e estruturas que permitem expansão sem alterar a lógica principal do sistema.

Um dos exemplos mais claros está no uso do enum *StatusSolicitacao*:

```
public enum StatusSolicitacao {  
    ABERTO(1, "Aberto", "🔴"),  
    TRIAGEM(2, "Triagem", "🟡"),  
    EM_EXECUCAO(3, "Em Execução", "🟠"),  
    RESOLVIDO(4, "Resolvido", "🟢"),  
    ENCERRADO(5, "Encerrado", "⬛");  
}
```

Esse enum centraliza todos os estados possíveis de uma solicitação, incluindo informações associadas como código, label e ícone. A principal vantagem aqui é que, caso seja necessário adicionar um novo status (por exemplo, “Em análise”), basta incluir um novo valor no enum, sem precisar modificar a lógica das classes que utilizam esse status.

Isso funciona porque o restante do sistema depende da abstração do enum, e não de valores fixos espalhados pelo código. Um exemplo disso está no método:

```
public static StatusSolicitacao fromCodigo(int codigo)
```

Esse método encapsula a lógica de conversão, evitando que o restante do sistema precise conhecer detalhes internos do enum. Assim, ao adicionar novos status, não é necessário alterar múltiplos pontos do código.

Outro exemplo de aplicação do *Open/Closed Principle* aparece no uso do enum *Categoria*:

```
public enum Categoria {  
    INFRAESTRUTURA_URBANA,  
    ILUMINACAO_PUBLICA,  
    LIMPEZA_URBANA,  
    ...  
}
```

A criação de solicitações utiliza diretamente esse enum. Isso permite que novas categorias sejam adicionadas sem alterar a lógica de criação de solicitações nem os métodos do *service*. O sistema automaticamente passa a suportar a nova categoria.

O mesmo acontece com o enum *Prioridade*, que define os níveis possíveis:

```
public enum Prioridade {  
    BAIXA,  
    MEDIA,  
    ALTA  
}
```

Toda a lógica de ordenação por prioridade, por exemplo em:

```
public List<Solicitacao> listarPorPrioridade()
```

continua funcionando independentemente da quantidade de valores definidos no enum. Ou seja, o comportamento do sistema é extensível sem necessidade de modificação.

A aplicação do OCP no restante do sistema melhora a manutenção porque reduz a necessidade de alterações em código já existente. Isso diminui o risco de introdução de bugs e facilita a evolução do sistema ao longo do tempo. Por exemplo, novas categorias, prioridades ou status podem ser adicionados sem impactar diretamente as regras de negócio ou as entidades principais.

3. Liskov Substitution Principle

O princípio da substituição de Liskov define que objetos de uma classe derivada devem poder substituir objetos da classe base sem alterar o comportamento esperado do sistema. Ou seja, qualquer subclasse deve respeitar o contrato da superclasse, garantindo que o sistema continue funcionando corretamente independentemente da implementação concreta utilizada.

No sistema, esse princípio aparece principalmente na relação entre a classe abstrata *Usuario* e suas subclasses *Cidadao* e *ServidorPublico*.

A classe *Usuario* define a estrutura base comum para todos os tipos de usuários do sistema, contendo atributos como nome, email e o indicador de perfil administrativo:

```
abstract class Usuario {  
    private String nome;  
    private String email;  
    private Boolean isAdmin;  
}
```

A partir dessa classe, são criadas especializações com comportamentos específicos. A classe *Cidadao*, por exemplo, adiciona o atributo telefone:

```
public class Cidadao extends Usuario {  
    private String telefone;  
}
```

Já a classe *ServidorPublico* adiciona o atributo cargo:

```
public class ServidorPublico extends Usuario {  
    private String cargo;  
}
```

Apesar dessas diferenças, ambas as classes podem ser utilizadas de forma uniforme como *Usuario* dentro do sistema. Isso pode ser observado, por exemplo, na classe *Solicitacao*, que recebe um usuário sem se preocupar com o tipo específico:

```
public Solicitacao(Usuario usuario, Categoria categoria, String  
descricao, String localizacao, Prioridade prioridade)
```

Nesse caso, tanto um *Cidadao* quanto um *ServidorPublico* podem ser utilizados sem necessidade de adaptação ou tratamento especial. Isso mostra que as subclasses respeitam o contrato definido pela classe base, permitindo substituição sem impacto no comportamento.

Outro exemplo aparece na classe *SolicitacoesService*, no método:

```
public Solicitacao criarSolicitacao(Usuario usuario, Categoria  
categoria, String descricao, String localizacao, Prioridade  
prioridade)
```

Esse método também trabalha com o tipo *Usuario* de forma genérica, sem depender de uma implementação específica. Isso garante flexibilidade no uso das subclasses e evita acoplamento direto com classes concretas.

Além disso, o método de atualização de status também segue essa lógica:

```
public void atualizarStatus(String protocolo, StatusSolicitacao  
status, String comentario, Usuario usuario)
```

Aqui, qualquer objeto que seja um *Usuario* pode ser passado como responsável pela atualização, mantendo o comportamento consistente independentemente da subclasse utilizada.

A aplicação do *Liskov Substitution Principle* no restante do sistema melhora a manutenção ao permitir que novas variações de usuário sejam adicionadas sem impactar diretamente as classes que já utilizam o tipo *Usuario*. Desde que essas novas subclasses respeitem o contrato definido pela classe base, elas poderão ser utilizadas nos mesmos pontos do sistema sem necessidade de modificação. Isso reduz o acoplamento e facilita a evolução do código.

4. Interface Segregation Principle

O princípio da segregação de interfaces define que uma classe não deve ser obrigada a depender de métodos que não utiliza. Em outras palavras, ao invés de criar estruturas genéricas e grandes, o sistema deve ser dividido em partes menores e mais específicas, garantindo que cada classe utilize apenas o que realmente precisa.

No sistema desenvolvido, não há uso explícito de interfaces, porém o conceito do *Interface Segregation Principle* pode ser analisado a partir da forma como as responsabilidades foram separadas entre as classes e como cada parte do sistema acessa apenas os comportamentos necessários.

A classe *SolicitacoesService*, por exemplo, atua como ponto central das operações relacionadas às solicitações. Ela expõe apenas métodos específicos para cada ação do sistema:

```
public Solicitacao criarSolicitacao(...)
public List<Solicitacao> listarSolicitacoes()
public List<Solicitacao> listarPorPrioridade()
public List<Solicitacao> listarPorCategoria()
public List<Solicitacao> listarPorStatus()
public Solicitacao buscarPorProtocolo(String protocolo)
public void atualizarStatus(String protocolo, StatusSolicitacao
status, String comentario, Usuario usuario)
```

Cada método atende a uma necessidade específica, evitando a criação de um único método genérico para múltiplas operações. Isso permite que outras partes do sistema utilizem apenas o que precisam, sem depender de funcionalidades desnecessárias.

Na classe *Menu*, por exemplo, os métodos do *service* são utilizados de forma isolada conforme a funcionalidade escolhida pelo usuário. No fluxo de criação de solicitação:

```
s.criarSolicitacao(cidadao, categoria, descricao, localizacao,
prioridade);
```

Já no fluxo de busca:

```
Solicitacao solicitacao = s.buscarPorProtocolo(protocolo);
```

E no caso de listagem:

```
List<Solicitacao> lista = s.listarSolicitacoes();
```

Isso mostra que o sistema não força a classe *Menu* a depender de um conjunto único e complexo de operações. Em vez disso, ela acessa apenas os métodos necessários para cada contexto, o que reduz o acoplamento e melhora a clareza do código.

Outro ponto relevante está na classe *FilaAtendimento*, que também segue essa ideia ao oferecer métodos específicos para cada tipo de ordenação:

```
public List<Solicitacao> listarPorPrioridade()  
public List<Solicitacao> listarPorCategoria()  
public List<Solicitacao> listarPorStatus()
```

Cada método resolve um problema específico, evitando a necessidade de um método genérico com múltiplos parâmetros ou regras internas complexas. Isso torna o uso mais simples e evita que classes externas precisem conhecer detalhes de implementação.

A aplicação parcial do *Interface Segregation Principle* já contribui para a manutenção ao reduzir dependências desnecessárias e tornar o uso das funcionalidades mais direto. No entanto, uma aplicação mais completa, com a separação do *service* em interfaces menores (como criação, consulta e atualização), permitiria um acoplamento ainda menor e facilitaria a evolução do sistema, principalmente em cenários com múltiplos módulos ou integrações.

5. Dependency Inversion Principle

O princípio da inversão de dependência define que módulos de alto nível não devem depender de módulos de baixo nível, mas sim de abstrações. Além disso, as abstrações não devem depender de detalhes, e sim os detalhes devem depender das abstrações. O objetivo é reduzir o acoplamento entre as partes do sistema, facilitando mudanças e evolução do código.

No sistema desenvolvido, a aplicação desse princípio ocorre de forma parcial. É possível identificar alguns pontos onde há tentativa de desacoplamento, mas também existem dependências diretas entre classes que indicam que o *Dependency Inversion Principle* não foi plenamente aplicado.

A classe *SolicitacoesService*, por exemplo, representa um módulo de alto nível responsável pelas regras de negócio. No entanto, ela depende diretamente da implementação concreta da classe *FilaAtendimento*:

```
private FilaAtendimento fila;

public SolicitacoesService() {
    this.fila = new FilaAtendimento();
}
```

Nesse caso, o *service* cria diretamente a instância da fila, ficando acoplado a essa implementação específica. Isso significa que, caso seja necessário alterar a forma de armazenamento das solicitações (por exemplo, utilizar um banco de dados ou outro tipo de estrutura), será necessário modificar a própria classe *SolicitacoesService*, o que vai contra o princípio.

O ideal seria que essa dependência fosse feita por meio de uma abstração (como uma interface), permitindo que diferentes implementações pudessem ser utilizadas sem alterar o *service*. No estado atual, o código depende diretamente de um detalhe de implementação. Outro exemplo aparece na classe *Menu*, que depende diretamente da classe concreta *SolicitacoesService*:

```
private final SolicitacoesService s;

public Menu(SolicitacoesService s){
    this.s = s;
}
```

Apesar de a dependência ser injetada via construtor, o que já é um ponto positivo, ainda existe acoplamento com a implementação concreta, pois não há uso de abstração. Isso limita a flexibilidade do sistema, já que qualquer substituição do *service* exigiria mudanças no código do menu.

Por outro lado, existem pontos onde o sistema se aproxima do *Dependency Inversion Principle* ao trabalhar com tipos mais genéricos. Um exemplo disso está no uso da classe abstrata *Usuario*, que é utilizada em diversos métodos:

```
public Solicitacao criarSolicitacao(Usuario usuario, Categoria
categoria, String descricao, String localizacao, Prioridade
prioridade)
```

Nesse caso, o sistema depende de uma abstração (*Usuario*) ao invés de classes concretas como *Cidadao* ou *ServidorPublico*. Isso permite que diferentes tipos de usuário sejam utilizados sem alterar o comportamento das classes que consomem esse tipo.

Outro ponto semelhante ocorre no uso de enums como *Categoria*, *Prioridade* e *StatusSolicitacao*, que funcionam como formas de abstração para valores do sistema, evitando dependência de valores fixos espalhados pelo código.

Mesmo assim, a ausência de interfaces para componentes principais, como o gerenciamento de solicitações (*FilaAtendimento*) e as regras de negócio

(*SolicitacoesService*), mostra que o sistema ainda depende fortemente de implementações concretas.

A aplicação parcial do *Dependency Inversion Principle* impacta diretamente a manutenção. Como as classes estão acopladas a implementações específicas, qualquer mudança estrutural exige modificação em múltiplos pontos do sistema. Isso aumenta o risco de erros e dificulta a evolução do código. Por outro lado, onde o princípio é seguido, como no uso da abstração *Usuario*, o sistema se torna mais flexível e preparado para extensões futuras.

Uma aplicação mais completa do *Dependency Inversion Principle* permitiria desacoplar completamente as camadas do sistema, facilitando a substituição de implementações, testes e evolução da arquitetura sem impacto direto nas classes de alto nível.

Conclusão

A análise do código mostra que os princípios SOLID foram aplicados de forma parcial, mas já com uma base bem estruturada, principalmente na separação de responsabilidades e na organização das classes.

O *Single Responsibility Principle* é o princípio mais bem aplicado, com uma divisão clara entre entidades (*Solicitação*, *HistoricoStatus*), regras de negócio (*SolicitacoesService*) e controle de dados (*FilaAtendimento*). Isso já garante uma boa organização e facilita alterações pontuais no sistema.

O *Open/Closed Principle* também aparece de forma consistente no uso de enums (*StatusSolicitacao*, *Categoria*, *Prioridade*), permitindo que o sistema seja estendido sem necessidade de modificar a lógica principal. Isso torna o código mais preparado para mudanças futuras, especialmente na evolução das regras do sistema.

O *Liskov Substitution Principle* é respeitado na hierarquia de *Usuario*, permitindo que *Cidadao* e *ServidorPublico* sejam utilizados de forma genérica em diferentes partes do sistema. No entanto, o uso de verificações como *getAdmin()* mostra que o polimorfismo ainda não é totalmente explorado.

O *Interface Segregation Principle* é aplicado de forma indireta, através da criação de métodos específicos e bem definidos no *service* e na *fila*, evitando estruturas genéricas e complexas. Ainda assim, a ausência de interfaces indica que há espaço para melhorar a separação de responsabilidades entre contratos e implementações.

Já o *Dependency Inversion Principle* é o princípio menos aplicado, pois as classes ainda dependem diretamente de implementações concretas, como ocorre entre *SolicitacoesService* e *FilaAtendimento*. Isso aumenta o acoplamento e limita a flexibilidade do sistema.

De forma geral, o código apresenta uma boa base de organização e já segue vários conceitos importantes de design. No entanto, ainda existem pontos de acoplamento e centralização de responsabilidades que podem ser melhorados para tornar o sistema mais flexível, reutilizável e fácil de manter.

A evolução desses pontos, principalmente com o uso de abstrações e melhor separação de responsabilidades, tende a reduzir o impacto de mudanças futuras e aumentar a qualidade estrutural do sistema como um todo.